

Supporting Information for:

Bloch-McConnell Simulation Study – Comparison of Various CEST Simulation Frameworks

Markus Huemer¹, Patrick Schuenke^{2,3}, and the BMsim Challenge participants

¹Institute of Biomedical Imaging, Graz University of Technology, Graz, Austria

²Physikalisch-Technische Bundesanstalt (PTB), Braunschweig and Berlin, Germany

³NVision Quantum Technologies, Ulm, Germany

July 8, 2026

This Supporting Information provides pool-model parameters and additional implementation details for the participating simulation submissions referenced in the main manuscript.

1 Description of the pool models

Table 1: Parameters for the 2 pool model

Pool	f(1)	T₁(s)	T₂(s)	k (Hz)	dw (ppm)
water	1	3	2	-	-
creatine	5e-4	1.05	0.1	50	1.9

Table 2: Parameters for the 5 pool model

Pool	f(1)	T₁(s)	T₂(s)	k (Hz)	dw (ppm)
water	1	1	0.04	-	-
MT	0.1351	1	4e-5	30	-3
amide	9.009e-4	1	0.1	50	3.5
guanidine	9.009e-4	1	0.1	1000	2
NOE	4.5e-3	1.3	0.005	20	-3

2 Additional description of simulations

S01 – M. Zaiss – Pulseseq-CEST

The submission used the MATLAB implementation of Pulseseq-CEST [1], which combines a Pulseseq-based sequence parser with a C++ Bloch-McConnell simulation core. Pool parameters were read from the YAML model files provided with the challenge, and saturation schemes were simulated from the supplied Pulseseq `.seq` files. Magnetization evolution was computed sequentially for each sequence block (RF, delays, and gradients). For piecewise-constant time intervals, the affine Bloch-McConnell system was propagated using a Padé approximation to the matrix exponential. Pools were assembled dynamically from the YAML specification, including separate water, CEST, NOE, and full xyz MT compartments. Shaped RF pulses were resampled according to the maximum number of samples defined in the model file. The Pulseseq-CEST code and documentation are available at <https://pulseseq-cest.github.io/>.

S02 – P. Schuenke – BMCTool

BMCTool is a purely Python-based Bloch–McConnell simulation framework for CEST and related multi-pool exchange experiments. Simulations are driven by Pulseseq `.seq` files (parsed via PyPulseseq), which provide the full event timing (RF, gradients, delays, ADCs), while scanner settings and pool parameters are specified in YAML configuration files. The magnetization evolution is computed block-by-block: RF events are optionally resampled to a user-defined maximum number of samples and applied piecewise; delays are simulated as free precession/relaxation; and spoiler gradients are modeled as complete transverse spoiling. For each time interval, the affine Bloch–McConnell system $\dot{\mathbf{M}} = \mathbf{A}\mathbf{M} + \mathbf{c}$ is propagated using a Padé approximation to the matrix exponential (with the steady-state shift computed from $\mathbf{A}^{-1}\mathbf{c}$ via a pseudoinverse), enabling stable simulation of arbitrary Pulseseq-defined irradiation schemes.

S03 & S04 – M.Huemer & R.Stollberger and M.Huemer & R.Stollberger 2

Simulation based on work published in [2], written in MATLAB. The second submission does not use the operator splitting proposed in [2], as a check on the influence this has on the simulation results. As described in the paper, a predefined constant time-step is used, and all offsets are solved in parallel. This can lead to memory problems and limit accuracy with long saturation times and many offsets. In S03 in each case, the timestep was chosen such that all available RAM (64GB) of the test workstation was used. The timesteps were: [1e-4, 1e-5, 1e-5, 5e-8, 2e-7, 1e-5, 1e-5, 1e-7]. The simulation in S04 does not solve all offsets in parallel, therefore the memory demand is much lower and shorter time increments are possible, this can increase performance (visible in Case 3), but leads to longer computation times. The simulation and derivatives are used in [3][4] and the simulation source code is available in the data to reproduce these results <https://doi.org/10.5281/zenodo.15768062>. Scripts to reproduce the simulation results of S03 can also be found here: <https://gitlab.tugraz.at/ibi/mrirecon/miscellaneous/bmc-simulation-matlab>

S06 – N. Vladimirov & O. Perlman - open-py-cest-mrf

The simulator was built with a C++ backend, a Pulseseq sequence specification, and a Python front end that supports parallel computation. The backend closely follows the Pulseseq-cest implementation and provides an interface to facilitate MRF simulations, streamlining parameter modifications. The Bloch-McConnell equations are solved in an iterative manner using the Pade approximation for matrix exponentiation, with a SWIG-based wrapper that enables Python integration. The Python front end is tailored for MRF simulations, using BMCTool definitions to set up and modify parameters on the fly while efficiently handling C++ function calls [5].

S09 – D. L. Longo & F. Ramdhane

Simulation based on the Bloch-McConnell work published in [6], written in Matlab by adapting the code available at https://github.com/cest-sources/BM_sim_fit. The BM equations have been solved analytically.

S12 – H.Zhang & J.Huang - CESTsimu

CESTsimu is an open-source GUI-based CEST simulation tool, written in MATLAB. It is compatible with Pulseseq-CEST, able to simulate 1D/2D CEST under B_0/B_1 inhomogeneities. The simulation solver is implemented through numerical iteration of the Bloch-McConnell equations. RF events are defined using cell arrays, where each cell represent either a continuous wave or a shaped pulse. The solver processes each RF event sequentially. The time step is specified within each pulse cell. For each time interval, the evolution is modeled in matrix form as $d\mathbf{M}/dt = \mathbf{A}\mathbf{M} + \mathbf{c}$, and the equations are solved using two built-in MATLAB functions: `mldivide()` and `expm()`. The solver code along with its demo are available at: https://github.com/JianpanHuang/BMsim_challenge.

S14 – V. Buchegger, M. Juschitz, N. Scholand & M. Uecker – BART

The BART submission used the Bloch–McConnell simulation implementation in the Berkeley Advanced Reconstruction Toolbox (BART). The BART toolbox is a free and open-source image-reconstruction framework for Computational Magnetic Resonance Imaging [7]. The Bloch–McConnell simulation is written in C and solves the Bloch–McConnell equations using a Runge–Kutta–Dormand–Prince scheme [8]. The main submission (S14) used single-precision floating point arithmetic; an additional variant (S14b) was provided using double precision. The source code is available at <https://codeberg.org/mrirecon/bart>.

S15 – M. Quach – CESTSim.jl

CESTSim.jl provides a Julia-based implementation for Bloch–McConnell simulations, with flexible choices of ODE solvers afforded by DifferentialEquations.jl. For the submissions to this challenge, the ODE solver of choice was a 5th-order Rosenbrock–Wanner method [9]. To demonstrate the impact of the choice of solver, an additional submission derived from using a 2/3rd-order Rosenbrock method (*Rosenbrock23()*) was provided to Case 2 (submission S15b in Figure 3 of the main manuscript). For 5-pool cases, it was assumed that all solute pools underwent exchange with water and water only (i.e., no cross-exchange between non-water pools). The relative tolerance for cases 1-4 (cw) and cases 5-8 (pulsed) were $1e-3$ and $1e-5$, respectively. The default absolute tolerance of $1e-6$ was used for all cases. For cases 5-8, the RF trains were interpolated and scaled using a constant (0th-order) B-spline. For each frequency offset, the entire RF train including pulse delays were interpolated and the system was solved across the entire saturation and spoiling delay duration, instead of block-by-block. In general, CESTSim.jl supports multi-pool simulations and the construction of complex saturation and imaging sequence protocols (and flexible interpolation options) for simulation. Other conditions such as $\delta\omega$ shift, spoiling, and incomplete recovery can also be incorporated into the simulations.

S16 – F. Zimmermann et al. – MRpro

MRpro [10] is an open-source PyTorch-based framework for MR image reconstruction, quantitative parameter estimation, and deep learning. Its Bloch–McConnell simulator is GPU-accelerated and fully differentiable with respect to sequence and tissue parameters. It propagates the affine system $\dot{\mathbf{M}} = \mathbf{A}\mathbf{M} + \mathbf{c}$ via matrix-exponential evolution for each sequence block, falling back to an augmented homogeneous formulation when systems are singular. The simulator supports multi-pool exchange, static off-resonance, and relative B_1 inhomogeneity through modular blocks including constant RF, piecewise-shaped RF, delays, gradients with isochromat dephasing, spoiling, and readout. For piecewise-shaped pulses, the evolution is computed sample-by-sample with chunked propagation and activation checkpointing to reduce memory use, including in the long pulse-train benchmark cases 6 and 7.

S17 – O. Cohen

The simulator was implemented in pyTorch for differentiability and efficient GPU-usage. A helper script was used to interface between the YAML configuration files and the simulator parameters. Bloch–McConnell equations are solved using the pyTorch matrix exponential (`torch.matrix_exp()`) and `torch.linalg.solve()` for the steady-state term. The simulator supports multi-pool exchange for arbitrary number of pools and flexible acquisitions using user-defined acquisition schedules.

S19 – P. Parnianpour & E. P. Pioro

The BM equations ($dM/dt = A \cdot M + b$) were solved analytically using the matrix exponential: $M(t) = \expm(A \cdot t) \cdot (M_0 - M_{ss}) + M_{ss}$ where $M_{ss} = -A^{-1}b$ is the steady state (via `numpy.linalg.lstsq`). The matrix exponential uses `scipy.linalg.expm` (Al-Mohy & Higham

2009, scaling-and-squaring, Padé order selected automatically based on matrix norm up to $m=13$). No homogeneous augmentation was used. For shaped pulses (cases 5–7), the Gaussian envelope was sampled into 200 steps matching the .seq file. Case 8 uses block pulses with a $100\mu\text{s}$ interpulse delay. For pulse trains (cases 6–7: 36 pulses), the full-pulse propagator was precomputed once per offset and applied 36 times, reducing expm calls by $36\times$.

References

- [1] Herz Kai, Mueller Sebastian, Perlman Or, et al. Pulseseq-CEST: Towards multi-site multi-vendor compatibility and reproducibility of CEST experiments using an open-source sequence standard. *Magnetic Resonance in Medicine*. 2021;86:1845-1858.
- [2] Graf Christina, Rund Armin, Aigner Christoph Stefan, Stollberger Rudolf. Accuracy and performance analysis for Bloch and Bloch-McConnell simulation methods. *Journal of Magnetic Resonance*. 2021;329:107011.
- [3] Stilianu Clemens, Huemer Markus, Zaiss Moritz, Stollberger Rudolf. Generalization of Optimal Control Saturation Pulse Design for Robust and High CEST Contrast. *Magnetic Resonance in Medicine*. 2025;.
- [4] Huemer Markus, Stilianu Clemens, Scholand Nick, et al. Dynamic Transitions for Fast Joint Acquisition and Reconstruction of CEST-Rex and T1. *Magnetic Resonance in Medicine*. 2025;.
- [5] Vladimirov Nikita, Cohen Ouri, Heo Hye Young, Zaiss Moritz, Farrar Christian T., Perlman Or. Quantitative molecular imaging using deep magnetic resonance fingerprinting. *Nature Protocols* 2025 20:10. 2025;20:3024-3054.
- [6] Zaiss Moritz, Angelovski Goran, Demetriou Eleni, McMahon Michael T., Golay Xavier, Scheffler Klaus. QUESP and QUEST revisited – fast and accurate quantitative CEST experiments. *Magnetic Resonance in Medicine*. 2018;79:1708-1721.
- [7] Blumenthal Moritz, Schaten Philip, Mackner Daniel, et al. *mrrecon/bart: version 1.0.00*. 2026.
- [8] Scholand Nick, Wang Xiaoqing, Roeloffs Volkert, Rosenzweig Sebastian, Uecker Martin. Quantitative MRI by nonlinear inversion of the Bloch equations. *Magnetic Resonance in Medicine*. 2023;90:520-538.
- [9] Steinebach Gerd. Construction of Rosenbrock–Wanner method Rodas5P and numerical benchmarks within the Julia Differential Equations package. *BIT Numerical Mathematics* 2023 63:2. 2023;63:27-.
- [10] Zimmermann Felix Frederik, Schuenke Patrick, Aigner Christoph S., et al. MRpro - open PyTorch-based MR reconstruction and processing package. 2025;.